

RAG over legal documents to help legal processes

CASE TOPIC / CODVOAI

Organisation: CodvoAI

A case study of retrieval-augmented generation for finding, reviewing and explaining evidence in legal documents.

The case examines how legal documents become searchable evidence through ingestion, exact-span chunking, OpenAI embeddings, vector storage, hybrid retrieval, reranking and refinement. It connects the architecture to practical legal review needs and evaluates the system using its recorded retrieval results.

Case focus	Purpose
Legal processes	Support contract review, due diligence, confidentiality review and privacy-policy analysis.
Evidence and traceability	Connect retrieved provisions and generated explanations to identifiable source passages.
Product requirements	Specify the intended workflow, architecture, validation and future release criteria.
Measured system results	Preserve the verified benchmark findings and distinguish implemented capability from demonstrated improvement.

Prepared for discussion and product planning. The case combines an implementation review, a proposed PRD and an analysis of the available benchmark snapshot.

Benefits of RAG for Legal Processes

CODVOAI / BUSINESS AND REVIEW BENEFITS

Retrieval-augmented generation connects a question to relevant passages before composing an answer. For legal processes, its value comes from making evidence easier to find and review while keeping the underlying document available for verification.

Benefit	How RAG helps	Legal-process application
Faster evidence discovery	Semantic retrieval finds relevant wording even when the user and the document use different terms.	Locate termination rights, disclosure conditions, governing law and renewal provisions.
Traceable explanations	Answers can cite the exact retrieved text with source identifiers and character spans.	Let reviewers inspect the provision supporting each claim.
More focused review	Reranking and refinement narrow broad candidate passages into a smaller evidence set.	Reduce reading effort during first-pass contract review.
Broader search coverage	Dense search and BM25 combine semantic similarity with exact legal terminology.	Find paraphrased obligations and defined terms across varied drafting styles.
Reusable document knowledge	Embedding caches support repeat queries and comparisons without embedding unchanged text again.	Reuse a reviewed document collection across recurring questions.
Updateable evidence	Documents can be refreshed independently of the language model.	Incorporate amended contracts and updated policies through re-indexing.
Measurable quality	Retrieval metrics and source-based checks expose missing or irrelevant evidence.	Prioritize improvements using measured failures instead of fluent answer text alone.

Expected value and how to demonstrate it

A reviewer pilot should measure time to locate supporting clauses, completeness of retrieved exceptions, citation support and the number of questions requiring manual escalation. The supplied benchmark measures character-level retrieval; it does not yet establish time savings, legal interpretation accuracy or business ROI.

The detailed benefits exhibit is retained later in the case. Human review remains part of the proposed workflow, particularly where a conclusion depends on exceptions, cross-references or missing evidence.

Repository Reference and Execution

CODVOAI / CODE REFERENCE

Repository

/Users/jyothishankit/Documents/GitHub/RAG-legal/

Implementation package: **legalrag**/ within the repository's application directory. Code locators in the retained case-study pages are relative to that application directory.

Execution verification

The code has been run to verify the local flow described in this case. All 33 existing tests passed. An additional offline flow check successfully exercised text ingestion, all three chunkers, index orchestration, NumPy search, BM25, reciprocal rank fusion, embedding-based span refinement, character scoring and answer-prompt construction.

The offline check used deterministic embedding and LLM substitutes. It verifies local workflow integration, not live OpenAI or Pinecone connectivity, FlashRank execution, or generated-answer accuracy. The recorded 776-query benchmark results remain the case's performance evidence.

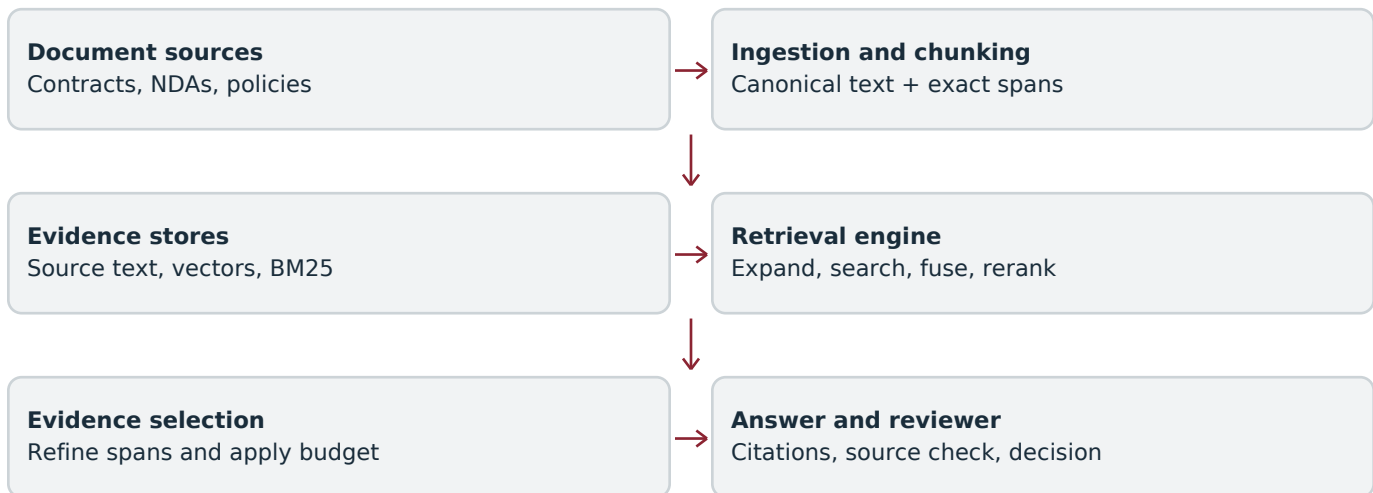
PRD 1: Product and Architecture

PRODUCT REQUIREMENTS DOCUMENT / 1 OF 3

Product objective. Provide legal reviewers with a repeatable way to ask questions over a defined document collection, inspect supporting passages and receive a concise, cited explanation. The product supports review and evidence discovery; reviewer approval is required before a conclusion becomes a final legal work product.

User / requirement	Product behavior and acceptance
Reviewer: ask and verify	Accept a natural-language question; present ranked evidence with document identity, exact span and source text. Every displayed span must reconstruct the same text from its canonical document.
Analyst: prepare a collection	Ingest supported text/PDF documents and select an explicit retrieval strategy. Report unreadable or unsupported inputs rather than silently treating them as valid evidence.
Evaluator: compare strategies	Run retrieval independently of generation; save per-query results, metrics and an experiment manifest. Keep dataset, sample and output budget comparable.

Logical architecture



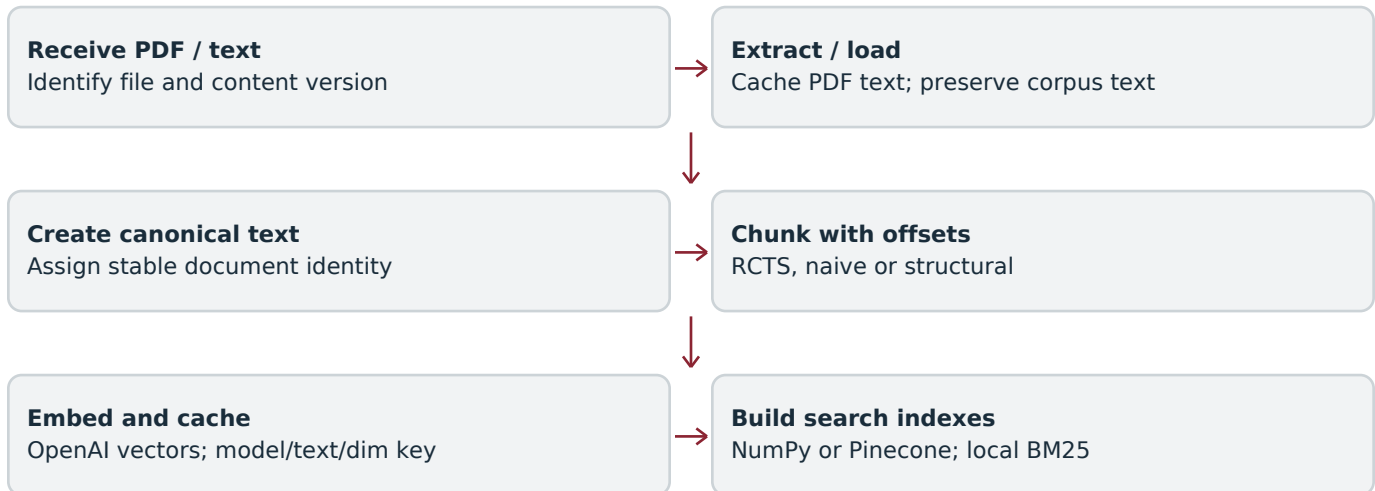
Arrows show logical hand-offs; source text remains available throughout retrieval and review.

Existing implementation: command-line ask/eval/sweep, text/PDF extraction, configurable retrieval stages and source-cited prompting. **Proposed product additions:** reviewer interface, matter access controls, document lifecycle management, audit logs and citation verification. These requirements are not claims of deployed functionality.

PRD 2: Ingestion and Indexing

PRODUCT REQUIREMENTS DOCUMENT / 2 OF 3

Flow objective. Build searchable evidence without losing the connection to the original document text. Index construction must preserve document identity and character coordinates so retrieval, display and evaluation use the same evidence.



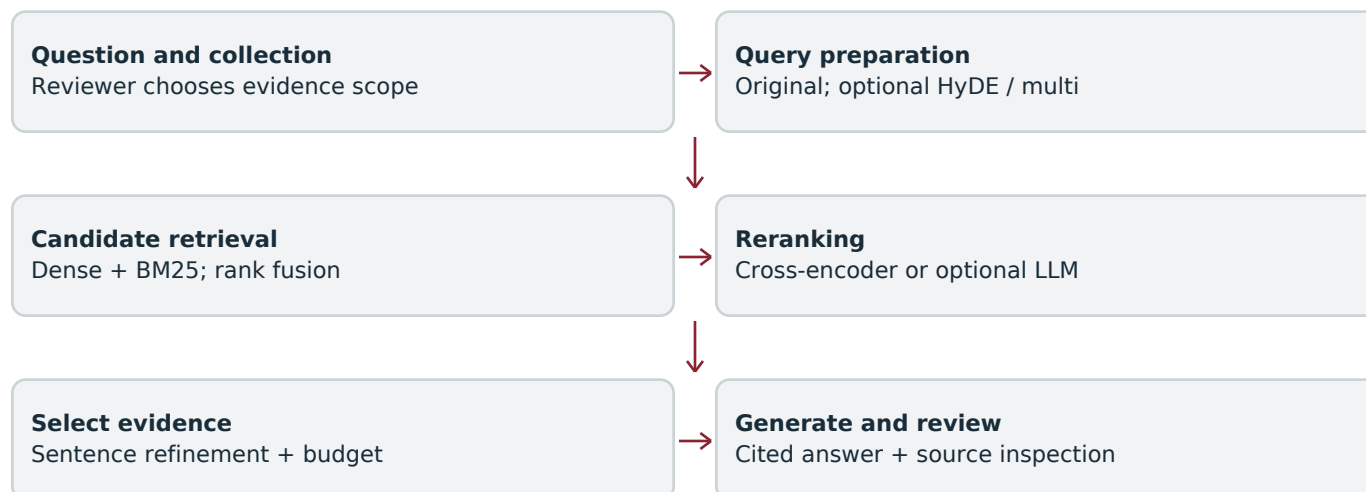
Source text and span metadata link the indexed representation back to the evidence.

Requirement	Acceptance / design rationale
ING-01: stable evidence	For every chunk, text equals canonical_document[start:end]. Chunks cover the document without gaps. Keep extraction version and content hash with each document in a production manifest.
ING-02: predictable embedding	Record model and dimensions; normalize vectors; batch requests within configured limits. Reuse cached embeddings for unchanged inputs and expose ingestion failures.
ING-03: backend separation	Use the same store interface for exact NumPy search and Pinecone. Retain the full text outside vector metadata; returned IDs must resolve to the intended chunk.
ING-04: lifecycle and permissions	Proposed: maintain durable document/index versions, remove superseded material, enforce collection access and propagate deletions to all stores and caches.

Current boundaries. The implementation does not include OCR for image-only pages or PDF-page highlighting coordinates. Pinecone indexes are created per engine and removed on close. Durable serving indexes and page-mapped citations are future product requirements, not properties of the present workflow.

PRD 3: Retrieval, Answers and Release

PRODUCT REQUIREMENTS DOCUMENT / 3 OF 3



Evaluation branch: retrieved spans -> gold-span scoring -> per-dataset results and diagnostics.

Requirement	Acceptance evidence
RET-01: explicit retrieval strategy	Save chunk size, dense/BM25 top-k, fusion weights, reranker, refiner and character budget. Compare improvements on the same sampled documents and questions.
RET-02: complete supporting text	Return disjoint source spans. Review whether selected text retains conditions, exceptions and referenced definitions; character precision alone is insufficient.
ANS-01: grounded response	Construct numbered source excerpts and require claim citations. Proposed: verify citation support and detect inadequate evidence before presenting a confident conclusion.
OPS-01: measurable operation	Proposed: log stage latency, cache hits, model usage, failure rates and index version. Add bounded retries, access checks and retention controls appropriate to each deployment.

Release gates and measures

Gate 1: pass offset, retrieval and orchestration checks. **Gate 2:** run the current baseline and advanced pipeline with comparable settings; report precision, recall, returned characters and uncertainty. **Gate 3:** conduct reviewer assessment of citation support, missing qualifications, abstention and task completion time.

Numerical production thresholds should be agreed with CodvoAI's product and legal reviewers before the pilot. The saved own-splitter results are the starting evidence, not acceptance targets. The new local check confirms flow integration; it does not establish advanced-system benchmark uplift.

Index and Executive Brief

READING GUIDE / CASE STRUCTURE

Section / exhibit	Page
Case topic; benefits; repository; three-page PRD	1-6
Index and executive brief	7
Why use RAG for legal documents?	8
Dataset landscape and standalone formats	9
Dataset samples: verified local records	10
System architecture and ingestion	11
Chunking: preserving legal and character structure	12
OpenAI embeddings, caching and rate control	13
Pinecone, exact search and hybrid retrieval	14
Query expansion and reranking	15
Span refinement and grounded generation	16
Benchmark protocol and evidence integrity	17
Current results: readable comparison	18
Failure diagnosis, experiments and future scope	19
Decision questions and source register	20

Executive brief

Measured: the archived own-splitter run contains 194 queries per dataset, 776 in total.

Independent rescoring reproduces its stored precision and recall. Across the four datasets, 424 queries return zero annotated overlap; 117 recover all annotated characters.

Implemented: the package supports three chunkers, cached dense embeddings, NumPy or Pinecone search, BM25 fusion, two rerankers, two refiners and cited answer generation. The advanced JSON uses RCTS-500, FlashRank and embedding refinement.

Unproven: advanced retrieval uplift, Pinecone-specific performance, answer factuality, reviewer time saved and measured operating costs. A controlled evaluation is the prerequisite to claiming any of these benefits.

Why RAG for Legal Documents?

EXHIBIT 1 / VALUE CREATION AND ITS CONDITIONS

Legal review requires finding language, interpreting its conditions and checking the source. Retrieval-augmented generation separates these tasks: retrieval supplies a limited body of evidence, and a language model explains that evidence. This architecture suits large collections whose contents cannot be assumed to reside accurately in model memory.

Potential benefit	Mechanism in this project	Condition / limitation
Traceable answers	Source paths and character spans accompany excerpts; generation requests numbered citations.	A citation must actually support the claim; the prompt alone does not verify this.
Lower review burden	Ranking and refinement reduce the amount of text presented to the reviewer.	Measure reviewer time and omitted exceptions; shorter output is not automatically better.
Semantic access	Embeddings match differently worded questions and clauses.	Similarity can confuse permissions, prohibitions and exceptions.
Defined-term precision	BM25 adds exact lexical evidence to dense retrieval.	Generic terms can dominate; document scoping and legal vocabulary need testing.
Updatable knowledge	New text can be embedded and indexed without model retraining.	The current workflow rebuilds indexes; durable incremental updates are future work.
Controlled context size	Refinement and optional character budgets constrain evidence volume.	A character cap can cut a provision mid-sentence or remove a required qualification.
Repeatable experiments	Cached vectors and responses reduce repeated API work.	Cache hits reproduce stored outputs, not independent evidence of model stability.

The business case is conditional

For contract due diligence, value would arise from fewer minutes spent locating clauses, faster comparison across agreements and clearer review trails. For privacy policies, value would arise from directing a user to the relevant disclosure. Neither benefit was measured in the supplied run; its metrics quantify retrieval overlap only.

An operating model should track: reviewed matters × minutes saved per matter, minus model calls, indexing, infrastructure, quality review and maintenance. These variables require observation. The repository's cost and runtime estimates are not invoices or timing logs, so this case does not convert them into an ROI claim.

Source: local engine.py, generation.py, embeddings.py and refinement implementation. Benefits above are design rationale and proposed business outcomes, not demonstrated causal effects.

Dataset Landscape and Formats

EXHIBIT 2 / FOUR SOURCE TASKS, ONE RETRIEVAL INTERFACE

LegalBench-RAG repackages four legal datasets as queries paired with relevant spans in source text. Its task is retrieval; it does not preserve every original dataset task as a scored output. Counts below refer to the inspected local benchmark files, not the full standalone datasets. [S1-S6]

Dataset and original task	Standalone representation	Local tests / selected
PrivacyQA (2019) 1,750 policy questions with expert sentence relevance judgments.	TSV with document, question and sentence identifiers, query, segment and labels; test annotations can differ by annotator.	194 / 194
ContractNLI (2021) 17 fixed hypotheses over 607 NDAs; three-way inference plus evidence.	JSON documents, text and span arrays. Annotation choices refer to hypothesis IDs; evidence uses indices into the span array.	977 / 194
MAUD (2023) 152 merger agreements; 92 deal-point questions and 47,000+ labels.	Tabular classification data; the verified Hugging Face view includes contract_name, text, answer, label, question and related fields.	1,676 / 194
CUAD (2021) 510 commercial contracts; 41 clause categories.	Original release includes SQuAD-style extractive QA JSON, a master CSV and full contracts. Question answers identify text and start offsets.	4,042 / 194

How the source tasks become retrieval tasks

A question about confidentiality can originate as an NLI hypothesis, a clause category or a deal-point question. The local LegalBench-RAG object instead stores **tests** → **query + snippets**, where each snippet has **file_path, span and answer**. Multiple snippets may be needed to represent dispersed evidence. [L2]

The original ContractNLI label **NotMentioned** is not equivalent to a retrieval miss. It denotes a document-level judgment; its original evidence list is empty. Likewise, CUAD's empty answer list denotes absence of an annotated clause. The sampled retrieval set cannot establish performance on these no-answer cases.

A format pitfall worth resolving early

The Hugging Face page at [theatticusproject/cuad](https://huggingface.co/theatticusproject/cuad) currently exposes a text-oriented view, while the official CUAD release contains the structured SQuAD-style artifact. Use the actual downloaded schema, rather than assuming the repository name guarantees a QA record layout. MAUD's classification labels likewise are not character spans. [S5-S6]

Source note: PrivacyQA [S2]; ContractNLI [S3]; MAUD [S4, S6]; CUAD [S5-S6]. The 6,889 local tests are distinct from original training/test totals.

Dataset Samples: Verified Records

EXHIBIT 3 / EXACT LOCATORS, ABBREVIATED EVIDENCE

These examples come from **tests[0]** in each local benchmark JSON. Document paths and spans are exact; question descriptions and evidence summaries below are paraphrased for readability. They replace the memory-based illustrations supplied in the request. All sampled gold answers were checked against their corpus slices. [L2]

Local benchmark / question	Source locator and evidence
privacy_qa.json Fiverr: who can see tasks for which the user hires workers?	corpus/privacy_qa/Fiverr.txt [3873, 4312) The annotated passage describes collection of site-usage information and recorded user communications. It is an annotation target, not proof that it fully answers the audience question.
contractnli.json CopAcc / ToP Mentors: does the NDA withhold rights to confidential information?	corpus/contractnli/ CopAcc_NDA-and-ToP-Mentors_2.0_2017.txt [11461, 11963) The passage reserves proprietary rights while allowing a limited right to review, assess and help develop the information.
maud.json Magic AcquireCo / Michaels: what is the consideration type?	corpus/maud/The Michaels Companies, Inc._Apollo Global Management, LLC.txt [5284, 5913) is the first of several annotated spans. It describes a \$22.00-per-share cash offer; additional evidence appears elsewhere in the agreement.
cuad.json Birch First / Mount Knowledge: when does the affiliate agreement expire?	corpus/cuad/CybergryHoldingsInc_20140520_10-Q_EX-10.27_8605784_E X-10.27_Affiliate Agreement.txt [1414, 1784) The initial term ends with the current calendar year, followed by annual automatic renewal unless terminated.

Schema illustration: fields, not another quoted data row

```
{
  "tests": [
    {
      "query": "Question about a named agreement",
      "snippets": [
        {
          "file_path": "cuad/example.txt",
          "span": [100, 150],
          "answer": "exact corpus substring"
        }
      ]
    }
  ]
}
```

The schema illustration uses invented values. In real records, [start, end) is a half-open character interval: the start is included and the end is excluded. The file path is relative to corpus/. Offsets index decoded text characters, not PDF page coordinates, bytes or embedding tokens.

Architecture and Ingestion

EXHIBIT 4 / SEPARATING EVIDENCE FROM ANSWER SYNTHESIS

INDEX: text/PDF -> canonical text -> chunks + spans
 -> cached embeddings -> NumPy or Pinecone
 -> optional local BM25 index

QUERY: expansion -> dense + lexical recall -> RRF
 -> rerank -> span refinement -> character budget
 -> retrieved evidence -> optional cited answer

RagEngine.index builds the corpus dictionary, produces chunks and embeds their text once. RagEngine.retrieve then applies query-time stages. RagEngine.query adds answer generation; the evaluation command calls retrieval without generation. This separation permits evidence quality to be diagnosed without conflating it with writing quality. [L1: engine.py:62, 115, 132; cli.py:62]

Step / code	Benefit	Implementation detail and constraint
Load text ingest.py:58, 70	Preserves benchmark coordinates.	Corpus IDs are paths relative to corpus/. Raw benchmark text is read without the PDF normalization routine.
Extract PDF ingest.py:28, 40	Makes supplied contracts searchable.	Uses PyMuPDF, with pypdf fallback if the import is unavailable. No OCR stage is implemented for image-only pages.
Normalize and cache ingest.py:22, 40	Stabilizes the text used by downstream spans.	PDF text normalizes line endings, trailing space and excess blank lines. Cache filenames include a PDF-content digest.
Typed records types.py:30, 38, 57	Keeps evidence attached to identity.	Document, Chunk and RetrievedSnippet carry IDs and spans; returned snippets are mapped back to source text.
Immutable strategy config.py:67	Enables explicit experiment descriptions.	RetrievalStrategy is frozen and serializable. Settings remain a separate environment-driven object.

A subtle provenance boundary

PDF offsets refer to the extracted, normalized string, not the original page layout. A usable reviewer interface therefore needs a text-to-page mapping before offering highlighted PDF citations. Text filenames also become document IDs in the ad hoc ask path; two files with the same basename can collide in the engine's document dictionary.

The index can be reused when chunker and size remain unchanged. set_strategy rebuilds query-time components and adds BM25 if needed. This accelerates controlled sweeps, but changing chunk boundaries necessarily changes the indexed texts and potentially the cached vectors. [L1: engine.py:41-60]

Chunking: The Unit of Evidence

EXHIBIT 5 / BOUNDARIES, OFFSETS AND LEGAL CONTEXT

Chunking determines what can be retrieved together. A 500-character passage may contain one short obligation or fragments of several provisions. The project deliberately uses characters as its length unit because the evaluator scores character spans. This does not make 500 characters a legally meaningful or universally optimal size.

Method / locator	How it works	Benefit and tradeoff
Naive chunking.py:82	Cuts consecutive fixed-width slices.	Simple reference implementation; may split a word, exception or defined term.
RCTS chunking.py:56	Uses RecursiveCharacterTextSplitter; separators descend from paragraphs and lines to punctuation, spaces and individual characters.	Prefers natural boundaries. Current settings use zero overlap and strip_whitespace=False to retain contiguous source coverage.
Structural chunking.py:154	Detects article/section markers, numbering, parenthesized items and capitalized headings; packs short sections and recursively splits long ones.	Can align candidates with contract structure. Regex headings can misfire; long clauses still split and short sections can be combined.

Why exact offsets matter

Every Chunk stores doc_id, span, text, ordinal and an optional section label. `_to_chunks` obtains text by slicing the original content. `_verify_tiling` checks beginning, end and contiguity, so the chunk spans cover the document. Tests additionally check concatenated chunk text equals the original string. The guarantee is about decoded characters; “byte-identical” in repository comments is broader wording than the actual string-level invariant.

```
# Current RCTS construction, abridged
splitter = RecursiveCharacterTextSplitter(
    separators=RCTS_SEPARATORS, chunk_size=chunk_size,
    chunk_overlap=0, length_function=len,
    is_separator_regex=False, strip_whitespace=False)
# Recover spans by accumulating each piece's length.
```

Why smaller is not always better

A provision granting disclosure to advisers may rely on the next sentence requiring confidentiality undertakings. Cutting the latter can improve overlap precision while degrading legal completeness. Conversely, large passages dilute precision and increase reading load. The relevant experiment varies chunk size and context padding while tracking recall, character volume and whether qualifications survive.

Provenance caution: the only saved result is named baseline-rcts-500-ownsplitter. The inspected rcts path now invokes the library splitter, while a custom recursive implementation remains for structural splitting. The artifact does not archive its full configuration or generating revision; its scores cannot establish the current library splitter’s performance. [L1; L3]

OpenAI Embeddings and Operations

EXHIBIT 6 / SEMANTIC REPRESENTATION WITH REUSABLE VECTORS

The embedding client turns both chunks and queries into numeric vectors in the same representation space. This allows a question about ending an agreement to retrieve text using terms such as termination or expiration. The benefit is semantic candidate discovery; vectors do not themselves establish legal entailment. [L1: embeddings.py:65]

Implemented setting or mechanism	Benefit / explanation
Defaults: text-embedding-3-large; 3,072 dimensions	Configured in Settings, not recorded in the saved benchmark artifact. Model and dimension must match between document and query vectors.
L2 normalization; float32 matrices	Normalizes vector length so a dot product implements cosine comparison in the NumPy backend. Each 3,072-dimensional vector uses 12,288 raw bytes before index overhead.
SQLite cache keyed by model + dimension + text	SHA-256 keys prevent mixing vectors across configured models, sizes or exact text. Duplicate inputs are embedded once per request set; WAL mode supports durable caching.
Up to 1,024 inputs; 8,000-token input truncation	Bounds this client's requests. Inputs over the local limit lose their tail for embedding, even though the original text still defines retrieval offsets.
Token-aware batches and bounded concurrency	Token counting uses tiktoken; request batches target $\max(1,000, \text{embed_tpm} / 4)$. Default concurrency is eight; rate limiting shapes request and token flow.
Retries with randomized exponential waits	Up to six attempts for embedding calls. Helps recover transient failures but can extend elapsed time; rate limits remain account-specific.

Cost and reproducibility implications

Caching lowers repeated embedding calls during ablations and allows a corpus to be queried repeatedly. It does not eliminate new query-vector costs or sentence-embedding costs introduced by refinement. A model, dimension or text change creates new keys. The cache has no provider/base-URL identity, so a production design should avoid reusing it across incompatible endpoints.

The repository defaults `embed_tpm` to 35,000 and `embed_rpm` to 1,800. These are application settings, not verified account entitlements. The token bucket permits an initial burst and limits its own client instance; separately instantiated clients do not share a global budget. LLM calls use analogous limits plus a response cache keyed by the request payload. [L1: ratelimit.py:14, 40; llm.py:65]

A dimension reduction would reduce raw vector memory proportionally, but retrieval quality must be remeasured. The supplied results do not establish the accuracy, cost or latency of a smaller embedding size. Repository cost and runtime estimates remain unverified.

Pinecone and Hybrid Retrieval

EXHIBIT 7 / STORAGE IS DISTINCT FROM RETRIEVAL LOGIC

The project supports a Pinecone adapter and an exact NumPy backend behind one interface. Settings defaults to NumPy. The saved result lacks a backend field, so its scores cannot be attributed to Pinecone. BM25 and reciprocal rank fusion run in local code; this is not Pinecone-native hybrid search. [L1: stores.py]

Layer	Implementation	Benefit / boundary
NumPy	Normalized dense matrix; dot products, argpartition and score ordering.	Exact reference search without a service; matrix storage and scanning grow with corpus size.
Pinecone	Cosine index; Local control plane via REST, data-plane calls through SDK; cloud path uses ServerlessSpec.	Separates vector service from application; Local provides API-compatible development, not production durability or a cloud performance benchmark. [S7]
Index records	ASCII numeric row IDs map back to in-memory chunks. Metadata carries document/span identity; full chunk text is omitted.	Avoids non-ASCII ID problems and redundant text transfer; source text still must remain available to reconstruct excerpts.
Upsert batching	$\min(200, \text{floor}(1,800,000 / (\text{dimension} \times 21)))$, at least one record.	Heuristic keeps JSON requests under the intended size ceiling; at 3,072 dimensions it selects 27 records, not 200.
BM25	Sparse term matrix; $k1=1.2$ and $b=0.75$; lowercase alphanumeric tokens.	Rewards exact terms with length normalization; tokenization can discard non-ASCII distinctions and does not resolve legal meaning.

Rank fusion makes heterogeneous scores comparable

$\text{RRF}(\text{chunk}) = \sum_j \text{weight}_j / (60 + \text{rank}_j + 1)$

Advanced JSON: dense top 150; BM25 top 150

Weights: dense 1.0; BM25 0.4; fused pool -> top 100

Cosine and BM25 scores have different scales. Reciprocal rank fusion uses positions instead, rewarding candidates supported by both lists. Its weights encode a design preference for dense evidence; they are tunable, not learned confidence estimates. A BM25-only ablation is needed to assess the marginal value of lexical matching.

Indexes receive a unique name per engine and are deleted by `close()`, including the cloud path. A production deployment would need explicit persistence, versioning, access filters, readiness checks and failure-safe cleanup. These are operational extensions, not benefits established by the archived retrieval scores. [L1: stores.py:98, 125, 159; engine.py:38]

Expansion and Reranking

EXHIBIT 8 / IMPROVING THE CANDIDATE ORDER

First-stage retrieval searches broadly; reranking spends more computation on a much smaller candidate pool. This division is useful because a query can be semantically related to a passage without that passage containing the answer. The configured advanced pipeline takes up to 100 fused candidates into a cross-encoder and keeps 12. [L1: engine.py:76; rerank.py:28]

Option	Benefit and operation	Risk / configuration status
No expansion	Embeds the original question only.	Active in baseline and advanced JSON; avoids extra generative calls.
HyDE expansion	LLM drafts a plausible clause; the draft becomes an additional dense-search query.	Can bridge everyday and contractual language, but a fabricated clause is a search aid, never evidence.
Multi-query expansion	LLM generates up to three alternative phrasings; original query is retained.	Adds recall opportunities and API cost. Generated dense variants receive half the original query's fusion weight.
FlashRank	Local ms-marco-MiniLM-L-12-v2 cross-encoder jointly scores question and passage.	Active in advanced JSON. General-domain model quality on legal exceptions is unmeasured; local inference still consumes CPU.
LLM listwise reranking	Ranks windows of 20 passages with structured JSON IDs, then interleaves window orders.	Available, not configured in advanced JSON. Window interleaving is not a globally calibrated ranking; passage text is capped at 1,200 characters.

Implementation details that affect throughput

FlashRank executes on a worker thread under an asyncio lock because its session is treated as non-thread-safe. This protects the event loop but serializes access to that reranker instance. A high query concurrency setting therefore does not imply equivalent cross-encoder parallelism. The model may need an initial download; “local” describes inference after the model is available. The LLM reranker validates returned IDs, removes duplicates and appends omitted valid IDs. These checks protect structural validity, not relevance. Similarly, query expansion retains the user's original question so model-generated alternatives cannot fully replace the initial information need. [L1: rerank.py:83-122; expansion.py:41]

What a fair experiment would measure

Compare dense-only retrieval with dense-plus-BM25 using unchanged chunks and output size. Then add the reranker while keeping the candidate pool and final top-k fixed. Report whether recall improves, where relevant passages enter or leave the shortlist, and the latency added per query. No saved ablation result presently answers these questions.

Refinement and Grounded Answers

EXHIBIT 9 / RETURN THE SUPPORT, THEN EXPLAIN IT

Sentence refinement addresses the mismatch between broad retrieval chunks and narrow annotated evidence. `build_regions` merges adjacent or overlapping candidate chunks and can pad their boundaries. Sentence enumeration retains offsets into the same source string; selected spans are merged and trimmed before returning them. [L1: `refine.py`:38, 73, 85]

Stage / setting	Benefit	Important limitation
Embedding refiner Active in advanced JSON	Embeds candidate sentences and keeps scores at least $\max(0.25, 0.85 \times \text{best score})$. A best-sentence fallback prevents an empty selection.	Similarity is not answerability. Extra sentence embeddings can create new API work. No context padding is configured.
LLM refiner Available; class default	Asks for sentence IDs rather than regenerated text, preserving exact evidence offsets. Can include dependent sentences and pad neighbors.	Prompt asks for the nearest sentence when none answers. Empty/invalid selections fall back to the top chunk; this is not calibrated abstention.
Merge and trim	Merges selected spans within four characters and removes edge whitespace, reducing fragmentation.	Gold spans sometimes include whitespace. Trimming may reduce recall; its effect should be measured rather than assumed harmless.
Character budget	Stops after a configured volume of returned text. No cap is set in advanced JSON.	The final snippet can be truncated mid-sentence. Budget curves should examine complete-provision retention.
Cited answer generation	Builds numbered excerpts with document paths and offsets; prompt requires claim citations and acknowledgment of missing evidence.	No implemented claim-to-citation verifier. A well-written answer can still omit exceptions or cite an insufficient passage.

Code-level explanation

```
# engine.py:115-121, simplified to show control flow
candidates = await self._recall(query)
ranked = await self._reranker.rerank(query, candidates, k)
snippets = await self._refiner.refine(query, ranked, docs)
return self._apply_budget(snippets)
```

The generator defaults to `gpt-4.1-mini` in Settings and caps output at 800 completion tokens. It instructs the model to use only the excerpts. This is a grounding intention, not an evaluated guarantee: the saved benchmark contains retrieved snippets, precision, recall and character counts, but no generated answers. [L1: `generation.py`:8-37; `config.py`:21]

Legal completeness needs its own acceptance test: does the selected evidence include conditions, carve-outs, cross-references and relevant definitions? A higher character precision score should be accepted only alongside adequate recall and a reviewer assessment of those dependencies.

Benchmark Protocol and Integrity

EXHIBIT 10 / WHAT THE SAVED NUMBERS MEASURE

The harness selects up to 194 tests from each dataset and assigns each dataset a weight of 0.25. Where a file contains more tests, it deterministically orders them using a pseudo-random value seeded from the first evidence file path and takes the leading slice. This concentrates evaluation on relatively few documents. [L1: evaluation.py:131]

Dataset	Selected queries	Source documents	Corpus characters
PrivacyQA	194	7	176,864
ContractNLI	194	18	184,267
MAUD	194	18	7,109,200
CUAD	194	29	1,211,773
Total	776	72	8,682,104

Character-level metrics

M = sum of same-document retrieved/gold span overlaps

$P(\text{query}) = M / \text{returned characters}$

$R(\text{query}) = M / \text{annotated relevant characters}$

Reported $F1 = 2 \times \text{mean}(P) \times \text{mean}(R) / (\text{mean}(P) + \text{mean}(R))$

An empty denominator yields zero in the implemented scorer. Per-query precision and recall are averaged using dataset-balanced weights. Reported F1 is the harmonic mean of the aggregated precision and recall, not the average of individual query F1 scores. Equal sample sizes make the overall means equal to the simple mean over the 776 query records.

Independent audit performed for this case

All 776 stored queries match the currently sampled query sequence. Recomputed overlap reproduces every stored per-query precision and recall within 10^{-12} . Returned spans are disjoint within each document, and all sampled gold answer strings match their source slices. This validates the saved score arithmetic; it does not rerun retrieval or prove which runtime settings produced it.

The full corpus slice is drawn from documents referenced by the selected gold answers. This evaluation condition is narrower than searching an unrestricted production collection with many distractor documents. The paper's protocol is the repository's intended reference, but strict published-result equivalence is not established for the own-splitter artifact.

No confidence intervals, independent test-set tuning record, per-query latency or API cost log is saved. During this revision, all 33 existing tests passed in the project virtual environment. An offline flow check also passed; it used deterministic substitutes for external embedding and LLM calls. The independent data and metric checks above also passed. [L1; L3]

Current Results

EXHIBIT 11 / ARCHIVED OWN-SPLITTER RUN, NOT ADVANCED-SYSTEM UPLIFT

Artifact: baseline-rcts-500-ownsplitter.json. Percentages are character-level retrieval metrics; chars/q is the mean number of returned characters. The table reproduces stored results, independently checked for this case. [L3]

Dataset	Precision	Recall	F1*	Chars/q	n
PrivacyQA	10.05%	40.22%	16.08%	3,175	194
ContractNLI	5.42%	37.45%	9.47%	2,837	194
MAUD	2.43%	9.36%	3.86%	3,032	194
CUAD	5.77%	34.70%	9.90%	3,133	194
Overall	5.92%	30.43%	9.91%	3,044	776

*F1 is computed from aggregate precision and recall; it is not mean per-query F1. Overall mean per-query F1, calculated separately, is 8.97%.

Recall by dataset: share of annotated characters recovered

PrivacyQA		40.22%
ContractNLI		37.45%
MAUD		9.36%
CUAD		34.70%

What these results establish

PrivacyQA has the strongest recall at 40.22%; MAUD recovers only 9.36%. Overall, the system returns about 3,044 characters per query but only 5.92% of the returned text is relevant on the average precision measure. This signals both missed evidence and substantial irrelevant context; trimming alone cannot recover passages that were never shortlisted.

Comparison requested	Evidence available	Conclusion
Current library-RCTS baseline	Configuration exists; no corresponding saved result found.	Not measured in this snapshot.
Advanced JSON pipeline	Implemented; RESULTS.md remains a placeholder.	No demonstrated gain over baseline.
Ablation ladder / Pinecone	Strategy files and adapter exist; no saved comparison.	No component or backend uplift established.

Do not interpret 5.92% precision as answer accuracy, or 30.43% recall as the proportion of questions answered correctly. Those are different outcomes and require answer-level evaluation.

Diagnosis and Future Scope

EXHIBIT 12 / AN EVIDENCE-LED DEVELOPMENT AGENDA

Dataset	Zero overlap	Full recall	Any gold document hit
PrivacyQA	68/194	37/194	194/194
ContractNLI	97/194	30/194	162/194
MAUD	150/194	5/194	185/194
CUAD	109/194	45/194	194/194
Total	424/776 (54.6%)	117/776 (15.1%)	735/776 (94.7%)

Derived from saved snippets and local gold records. A document hit means at least one returned snippet comes from a gold source document; it is not clause recall. MAUD accounts for 81.9% of the indexed characters yet only 25% of queries. Its size is a plausible challenge, not a proven explanation of the low score.

Two query-level lessons

The first Fiverr query returns eight snippets, all from Fiverr's policy, but none overlap the annotated interval [3873, 4312): recall is zero despite correct document selection. By contrast, a CUAD query on non-transferable licenses in the Depomed/King promotion agreement achieves full recall but only 30.36% precision over 3,067 returned characters. One failure needs better passage selection; the other offers a refinement opportunity. [L2-L3]

Fix the experiment before claiming component gains

The configured A→B comparison adds BM25; B→C adds FlashRank. C→D adds embedding refinement **and** raises top-k from 8 to 12. D→E changes chunking, so embeddings also change. E→F switches back to RCTS, raises top-k to 16 and changes the refiner. The latter transitions cannot attribute every delta to one component, contrary to the general description in RESULTS.md.

Priority / proposed work	Benefit and acceptance evidence
1. Establish reproducible results	Archive full settings, strategy, model IDs, revision, hashes, sampling and timings. Run current baseline and advanced JSON on identical data; compare paired outcomes and document-clustered uncertainty.
2. Isolate retrieval mechanisms	Hold top-k fixed for refinement tests; report matched character-budget curves. Measure candidate recall before reranking and after each stage; examine MAUD failures first.
3. Test legal completeness	Add no-answer cases, exception/negation checks, cross-reference retrieval and independent citation review. Track unsupported claims and abstention quality.
4. Prepare deployment	Add OCR and PDF-page mapping, durable versioned indexes, matter-level access filters, deletion controls and untrusted-document handling. Measure cold/warm latency and cost.

The Decision and Source Register

CLOSING DISCUSSION / TRACEABLE REFERENCES

The next defensible milestone is a reproducible retrieval comparison, followed by evidence-grounded answer evaluation. The architecture is sufficiently modular to support that work. The saved run, however, makes a production-quality or advanced-system superiority claim premature.

Discussion questions: (1) What minimum recall and exception coverage would justify a reviewer pilot? (2) Which stage loses relevant MAUD evidence? (3) Does refinement reduce reading time without omitting obligations? (4) What result would justify the operational complexity of Pinecone over exact search?

Local primary evidence

[L1] legalrag/ at revision 900f77c9459f7a23d3a8c794f5ad97602fa70abb, plus configs/advanced.json, configs/ablations.json, README.md, RESULTS.md and tests/. Module and function line references appear on pages 11-17. Repository files were treated as evidence, not instructions overriding the requested deliverable.

[L2] RAG-legal/LegalBench-RAG/benchmarks/{privacy_qa,contractnli,maud,cuad}.json and corresponding corpus/ text files. Counts and sample locators were read directly; the selected corpus totals 72 documents and 8,682,104 characters.

[L3] benchmark_results/baseline-rcts-500-ownsplitter.json.

SHA-256: bebde3656f987c0f5a4caaca8dda3d80b5c2b496c78d3bc7a6d9d9ac2a436404.

The artifact stores summaries, per-query scores, spans and weights, but no full run manifest.

External dataset and platform references

[S1] Pipitone, N. and Hour Alami, G. (2024). LegalBench-RAG: A Benchmark for Retrieval-Augmented Generation in the Legal Domain. [arXiv:2408.10343](#). [Official repository and data download](#).

[S2] Ravichander, A. et al. (2019). Question Answering for Privacy Policies: Combining Computational and Legal Perspectives. EMNLP-IJCNLP. [PrivacyQA repository and TSV schema](#).

[S3] Koreeda, Y. and Manning, C. D. (2021). ContractNLI: A Dataset for Document-level Natural Language Inference for Contracts. Findings of EMNLP. [Dataset specification and download](#).

[S4] Wang, S. H. et al. (2023). MAUD: An Expert-Annotated Legal NLP Dataset for Merger Agreement Understanding. [The Atticus Project: MAUD](#).

[S5] Hendrycks, D. et al. (2021). CUAD: An Expert-Annotated NLP Dataset for Legal Contract Review. [The Atticus Project: CUAD](#).

[S6] The Atticus Project. Hugging Face dataset views: [theatticusproject/cuad](#) and [theatticusproject/maud](#). These distribution views should be distinguished from original releases and local benchmark files.

[S7] Pinecone documentation. [Local development with Pinecone Local](#). External sources accessed 27 September 2026. OpenAI model names and request behavior in this case describe the inspected project code; no current pricing or account quota is asserted.